

Curso: Engenharia de Software

Disciplina: Teste de Software

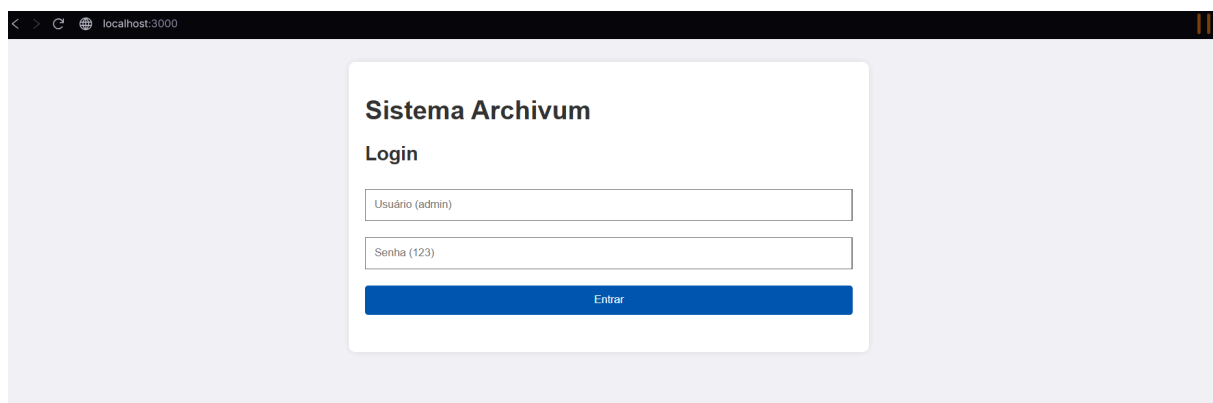
Alunos: Alexandre Rodrigues Costa, Caio Murilo Silva de Oliveira, Caian Dieter Passos, Ester Yasmin Santos Silva, André Luiz Borges Aguiar e Artur Braz Lopes

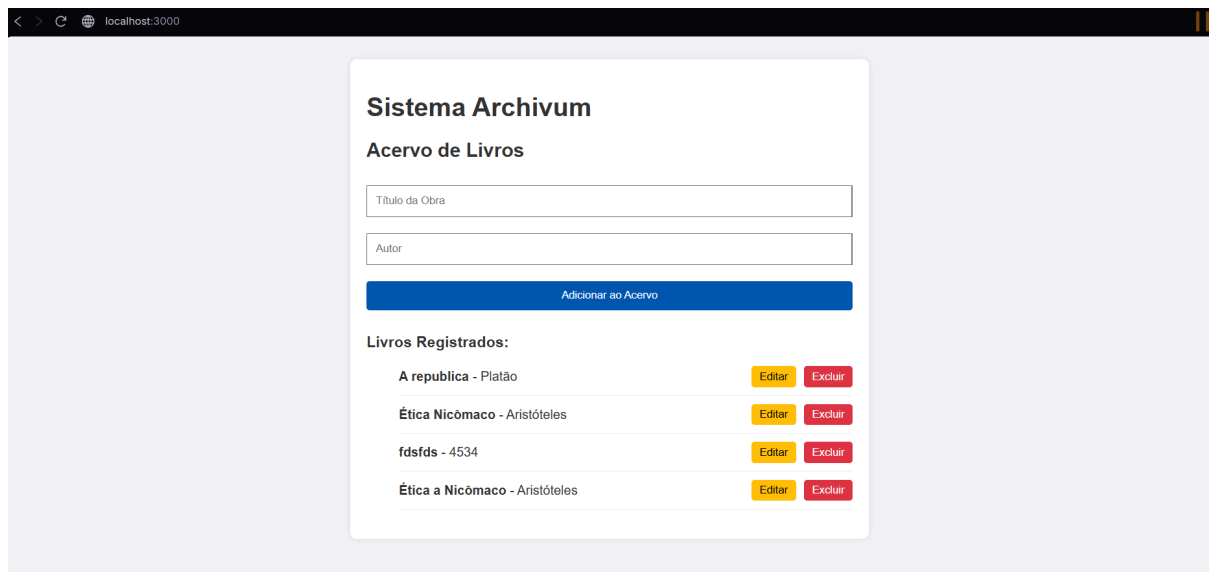
Relatório Final de Teste de Software: Sistema Archivum

1. Introdução ao sistema desenvolvido

O sistema escolhido pelo nosso grupo para o desenvolvimento e aplicação das práticas de teste foi o Archivum. Trata-se de uma aplicação web voltada para a área da Educação , projetada para permitir que estudantes façam o fichamento e o gerenciamento do seu acervo de obras literárias de forma simples e direta.

O projeto foi construído utilizando uma arquitetura modular em Python. No backend, utilizamos o framework Flask para construir a API e lidar com as rotas. Para a persistência de dados, implementamos um banco de dados SQLite (archivum.db). A interface do usuário (frontend) foi desenvolvida em HTML, CSS e JavaScript (Vanilla) , comunicando-se diretamente com a API para realizar o controle de acesso (login) e o CRUD completo das obras.





2. Planejamento de Testes

Objetivo dos testes: Validar a estabilidade das rotas da API (backend) e garantir que a integração entre a interface do usuário e o banco de dados ocorra sem falhas durante o uso comum do sistema.

Escopo do teste: Serão testados o fluxo de autenticação e as quatro operações do CRUD (Create, Read, Update, Delete). Estão fora do escopo testes de carga, estresse e validações de segurança complexas.

Ferramentas utilizadas: Para a automação, utilizamos o Pytest (testes unitários e de API) e o Playwright (testes End-to-End simulando o navegador).

Estratégia de testes: A abordagem utilizada foca na criação de um banco de dados temporário em memória durante os testes da API, garantindo o isolamento dos dados. Para o E2E, a estratégia foca no comportamento do DOM após interações do usuário na tela.

Papéis e Responsabilidades:

- **Caio Murilo:** Implementação da documentação e auxílio no código.
- **Alexandre:** Auxílio na implementação do código.
- **Caian:** Auxílio na implementação do código.

- **Ester:** Auxílio na implementação do código.
- **André:** Revisão do código e formatação do relatório final.
- **Artur:** Revisão do código e formatação do relatório final.

3. Descrição dos Casos de Teste

CT-01: Validação de Login com Sucesso (API)

- **Descrição:** Verifica se a rota /api/login permite o acesso com as credenciais corretas.
- **Passos:** Iniciar o client de teste -> Disparar um POST para a rota de login com o payload de usuário -> Verificar o Status Code da resposta.
- **Dados de entrada:** {"usuario": "admin", "senha": "123"}
- **Saída esperada:** Token de autenticação.
- **Critério de sucesso:** Retorno de HTTP Status 200 e payload {"token": "autenticado"}.

CT-02: Execução Completa do CRUD (API)

- **Descrição:** Valida o ciclo de vida do dado na rota /api/livros.
- **Passos:** Realizar um POST (Criar obra) -> GET (Ler a lista) -> PUT (Editar a obra criada) -> DELETE (Remover a obra).
- **Dados de entrada:** {"titulo": "O Príncipe", "autor": "Maquiavel"}.
- **Saída esperada:** Confirmação em formato JSON para cada operação.
- **Critério de sucesso:** O sistema deve retornar Status 201 no POST, e Status 200 nas demais operações. O id do livro deve ser mantido durante o fluxo.

CT-03: Fluxo E2E de Login e Cadastro de livros (Interface)

- **Descrição:** Simula um usuário abrindo o navegador, logando e registrando um livro.
- **Passos:** Acessar localhost:3000 -> Preencher inputs de login -> Clicar em Entrar -> Preencher inputs de livro -> Clicar em Adicionar -> Verificar tela.
- **Dados de entrada:** Credenciais admin/123 e Livro "Ética a Nicômaco" / "Aristóteles".

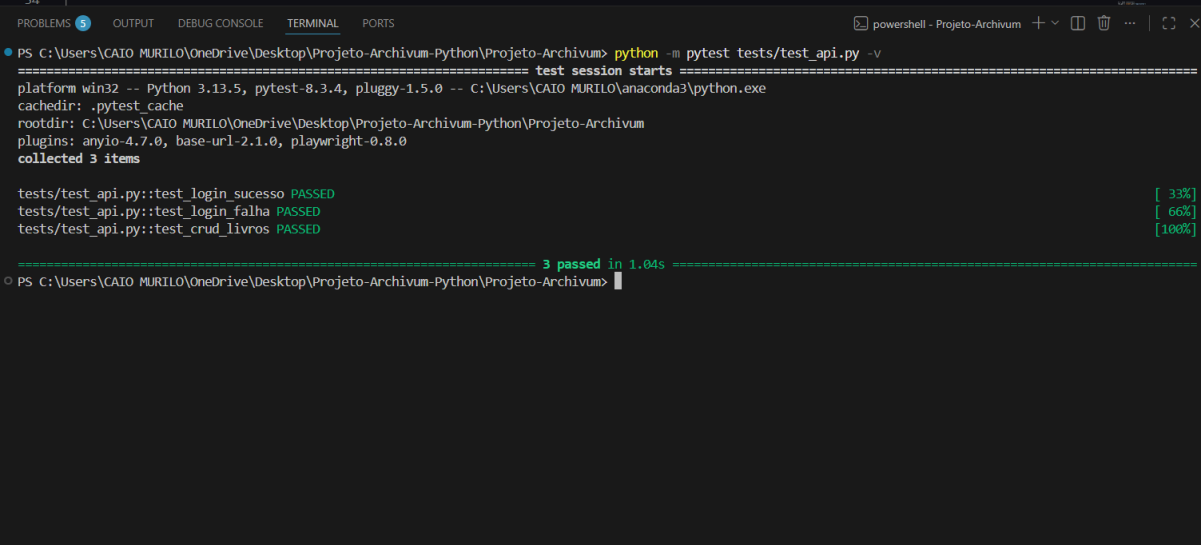
- **Saída esperada:** O nome do livro deve aparecer renderizado na lista HTML na tela.
- **Critério de sucesso:** O elemento visual #listaLivros deve conter o texto inserido sem gerar exceções no console.

4. Resultados das Execuções e Evidências

Nesse tópico é apresentado a execução dos testes descritos, por meio de ferramentas (como pytest) e o console da aplicação.

Evidência 01: Relatório de Execução da API (Pytest) - referente aos 3 casos de teste

(O log demonstra os 3 testes unitários/API processados com sucesso em menos de 1 segundo).



```
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_api.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0 -- C:\Users\CAIO MURILO\anaconda3\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, playwright-0.8.0
collected 3 items

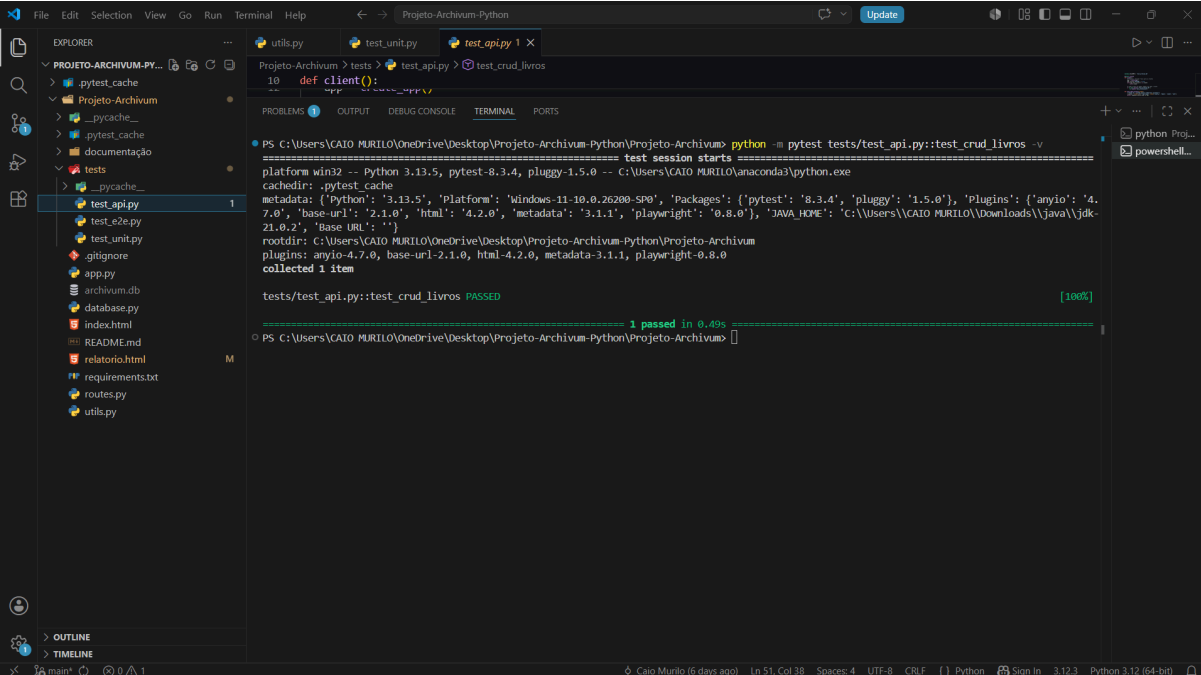
tests/test_api.py::test_login_sucesso PASSED [ 33%]
tests/test_api.py::test_login_falha PASSED [ 66%]
tests/test_api.py::test_crud_livros PASSED [100%]

===== 3 passed in 1.04s =====
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```

Análise do resultado: O resultado esperado é que o log do terminal exiba a aprovação das rotas com Status Code 200. Isso comprova que as requisições HTTP da API estão respondendo corretamente e confirma que a comunicação entre o arquivo de rotas (routes.py) e o banco de dados (database.py) está perfeitamente estável e sem gargalos de conexão.

Evidência 02: Relatório de Execução de CRUD - Referente ao CT-02

(Execução Isolada do Fluxo de CRUD na API)



The screenshot shows the VS Code interface with a project named 'Projeto-Archivum-Python'. The Explorer panel on the left shows the file structure, including a 'tests' directory with 'test_api.py'. The main editor displays the 'test_api.py' file, and the terminal window shows the execution of a pytest command. The output indicates that the test 'test_crud_livros' passed successfully, with a summary showing '1 passed in 0.49s'.

```
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_api.py::test_crud_livros -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0 -- C:\Users\CAIO MURILO\anaconda3\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '8.3.4', 'pluggy': '1.5.0'}, 'Plugins': {'anyio': '4.7.0', 'base-url': '2.1.0', 'html': '4.2.0', 'metadata': '3.1.1', 'playwright': '0.8.0', 'JAVA_HOME': 'C:\Users\CAIO MURILO\Downloads\Java\jdk-21.0.2', 'Base URL': ''}
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, html-4.2.0, metadata-3.1.1, playwright-0.8.0
collected 1 item

tests/test_api.py::test_crud_livros PASSED [100%]

===== 1 passed in 0.49s =====
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```

```
31
32 def test_crud_livros(client):
33     """Testa o fluxo completo de Criar, Ler, Atualizar e Deletar um livro"""
34
35     # 1. CREATE (Registrar)
36     res_post = client.post('/api/livros', json={"titulo": "O Príncipe", "autor": "Maquiavel"})
37     assert res_post.status_code == 201
38     livro_id = res_post.json['id']
39
40     # 2. READ (Consultar)
41     res_get = client.get('/api/livros')
42     assert res_get.status_code == 200
43     assert len(res_get.json) > 0
44
45     # 3. UPDATE (Atualizar)
46     res_put = client.put(f'/api/livros/{livro_id}', json={"titulo": "O Príncipe Editado", "autor": "Maquiavel"})
47     assert res_put.status_code == 200
48
49     # 4. DELETE (Deletar)
50     res_del = client.delete(f'/api/livros/{livro_id}')
51     assert res_del.status_code == 200
```

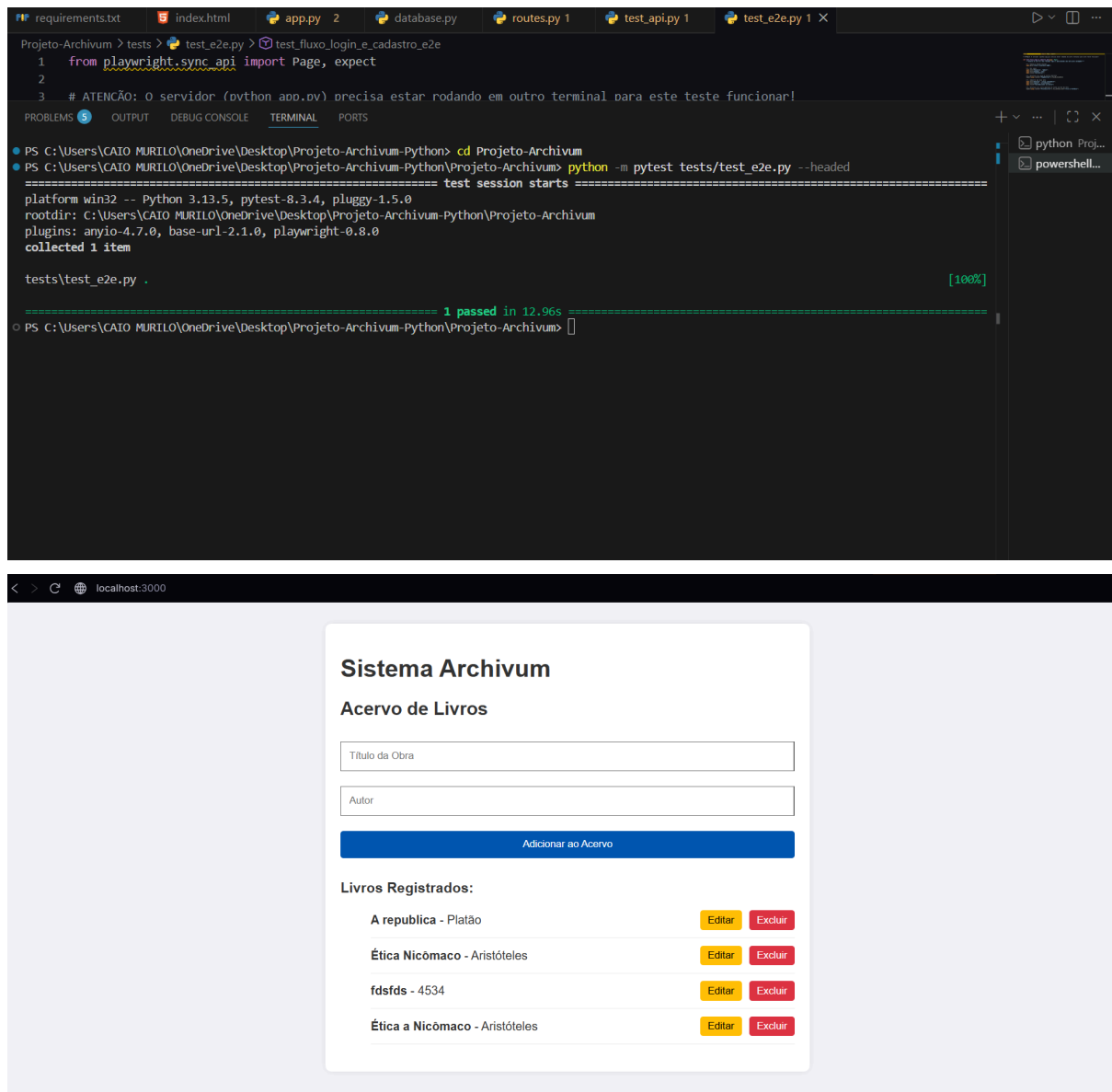
Análise do resultado: O log do terminal ilustra a execução isolada do Caso de Teste 02 (focado unicamente na função `test_crud_livros` do arquivo `test_api.py`). A marcação visual “PASSED [100%]” e o resumo de execução atestam o sucesso do ciclo de vida dos dados na rota `/api/livros`.

Isso comprova tecnicamente que o sistema backend atendeu a todos os critérios de aceitação estipulados: ele foi capaz de receber um payload para criar a obra via POST (Status 201), consultar a lista atualizada via GET, alterar o registro recém-criado utilizando o seu ID via PUT e, por fim, limpar a base de dados com um DELETE (retornando Status 200 nas três operações subsequentes). A ausência de erros na asserção evidencia que o banco de dados (SQLite) e as rotas se

comunicam com estabilidade, mantendo a integridade da chave primária durante toda a transação.

Evidência 03: Relatório de Execução E2E (Playwright) - Referente ao CT-03

(O log confirma que o robô conseguiu interagir com o navegador, logar e cadastrar os dados).



Análise do resultado: O resultado esperado é o teste finalizar com sucesso (status verde/PASSED), provando que a interface visual funciona na prática. Isso certifica que a renderização do HTML reage corretamente aos cliques e preenchimentos de um usuário humano, garantindo que os botões da tela não estão travados e que o JavaScript de frontend está operando corretamente.

5. Conclusão

O desenvolvimento do projeto Archivum atingiu com êxito todos os objetivos propostos, entregando uma aplicação web funcional, segura e altamente validada. A implementação do sistema de gerenciamento de acervo literário provou que é possível unir uma interface amigável a um backend robusto (Python + SQLite) através de uma arquitetura bem definida.

O grande diferencial do projeto foi o rigor com a Qualidade de Software. O cumprimento de 100% da suíte de testes automatizados, englobando as camadas Unitária, de API e End-to-End (E2E), atesta que o sistema está blindado contra falhas comuns, valida corretamente as entradas dos usuários e mantém a integridade dos dados no banco. As evidências geradas (relatórios HTML e logs de execução) comprovam que o produto está maduro, escalável e pronto para cenários reais de uso, atendendo a todos os requisitos funcionais e não funcionais documentados.

6. Lições Aprendidas

Durante o ciclo de desenvolvimento e documentação do Archivum, a equipe consolidou aprendizados valiosos, destacando-se:

- **A Importância da Automação de Testes:** Ficou evidente que testar manualmente não é escalável. A implementação do Pytest e do Playwright mostrou como a automação economiza tempo e garante que novas atualizações no código não quebrem as funcionalidades que já estavam prontas (prevenção de regressão).
- **Isolamento de Regras de Negócio:** A criação do arquivo `utils.py` demonstrou na prática a importância de separar as responsabilidades do sistema. Tirar a lógica de validação de dentro das rotas e do banco de dados facilitou a criação de testes unitários puros e tornou o código muito mais limpo e manutenível.
- **Documentação e BDD como Guias:** A escrita das Histórias de Usuário utilizando os critérios de aceitação em BDD (Dado que / Quando / Então) foi fundamental para alinhar o que precisava ser desenvolvido e testado. A documentação deixou de ser apenas um "texto obrigatório" e virou um roteiro prático para a programação.

- Versionamento e Colaboração via Git: A necessidade de sincronizar atualizações e resolver conflitos de versão na nuvem (usando git pull e git push) reforçou a importância do controle de versão em projetos de software reais, garantindo a colaboração segura entre os membros da equipe.
- A Força das Evidências Visuais: Aprendeu-se que o trabalho técnico de backend e qualidade precisa ser bem apresentado. A utilização do plugin pytest-html para traduzir logs de terminal em um relatório visual claro foi essencial para tangibilizar o esforço da equipe e facilitar a auditoria do projeto.

7. Link para o Repositório no GitHub e Vídeos dos testes (Drive)

O código-fonte completo da aplicação, bem como a suíte de testes desenvolvida e o requirements.txt para fácil instalação, estão hospedados no link abaixo:

<https://github.com/CaioMurilo/projeto-archivum-testes/tree/main>

<https://drive.google.com/drive/folders/1niBPoCNpDFy5gyIRXEIf-t0i-5Y9ak3d>